```
"product".camelize    # => "Product"
"admin_user".camelize # => "AdminUser"
```

As a rule of thumb you can think of this method as the one that transforms paths into Ruby class or module names, where slashes separate namespaces:

```
"backoffice/session".camelize # => "Backoffice::Session"
```

For example, Action Pack uses this method to load the class that provides a certain session store:

```
# action_controller/metal/session_management.rb
def session_store=(store)
  @@session_store = store.is_a?(Symbol) ?
    ActionDispatch::Session.const_get(store.to_s.camelize) :
    store
end
```

`camelize` accepts an optional argument, it can be `:upper` (default), or `:lower`. With the latter the first letter becomes lowercase:

```
"visual_effect".camelize(:lower) # => "visualEffect"
```

That may be handy to compute method names in a language that follows that convention, for example JavaScript.

As a rule of thumb you can think of `camelize` as the inverse of `underscore`, though there are cases where that does not hold: `"SSLError".underscore.camelize` gives back `"SslError"`. To support cases such as this, Active Support allows you to specify acronyms in `config/initializers/inflections.rb`:

```
ActiveSupport::Inflector.inflections do |inflect|
  inflect.acronym 'SSL'
end

"SSLError".underscore.camelize # => "SSLError"
```

`camelize` is aliased to `camelcase`.

Defined in `active_support/core_ext/string/inflections.rb`.

5.11.4 `underscore`

The method `underscore` goes the other way around, from camel case to paths:

```
"Product".underscore   # => "product"
"AdminUser".underscore # => "admin_user"
```

Also converts "::" back to "/":

```
"Backoffice::Session".underscore # => "backoffice/session"
```

and understands strings that start with lowercase:

```
"visualEffect".underscore # => "visual_effect"
```

`underscore` accepts no argument though.

Rails class and module autoloading uses `underscore` to infer the relative path without extension of a file that would define a given missing constant:

```
# active_support/dependencies.rb
def load_missing_constant(from_mod, const_name)
  ...
  qualified_name = qualified_name_for from_mod, const_name
  path_suffix = qualified_name.underscore
  ...
end
```

As a rule of thumb you can think of `underscore` as the inverse of `camelize`, though there are cases where that does not hold. For example, `"SSLError".underscore.camelize`gives back `"SslError"`.

Defined in `active_support/core_ext/string/inflections.rb`.